# CSE 400 Lecture 10 scribe

**L10_S2.pdf**
PDF

Generate a lecture scribe intended to serve as exam-oriented reference material for CSE 400: Fundamentals of Probability in Computing.

Use only the provided context, which consists of the lecture slides/PDF for Lecture 10 and the relevant textbook material explicitly associated with this lecture. Do not use any external knowledge and do not introduce material that does not appear in the provided context.

The lecture scribe must faithfully reconstruct what was taught in class. Include all definitions, notation, assumptions, statements of results or theorems, algorithm descriptions, pseudocode, and worked examples exactly as they appear in the provided context. Proofs or derivations must be included only if they are present in the lecture material.

Organize the scribe clearly using section headings that follow the structure and flow of the lecture. Maintain a formal academic tone suitable for exam preparation. Ensure logical continuity so that the document can be reliably used for revision.

Do not simplify, reinterpret, or add intuition beyond what is explicitly presented in the provided context. Do not introduce new examples, explanations, commentary, or material beyond the lecture scope.

Below is an **exam-oriented lecture scribe for Lecture 10** constructed strictly from the provided slides and associated material. All definitions, statements, pseudocode, and terminology follow the lecture content and sequence.

---

# CSE 400: Fundamentals of Probability in Computing

## Lecture 10: Randomized Min-Cut Algorithm

Instructor: Dhaval Patel

Date: February 5, 2026

Source: Lecture slides (Lecture 10)

---

# 1. Overview of the Lecture

The lecture covers:

- Min-Cut Problem
- Why use min-cut
- What is min-cut
- Successful min-cut run
- Unsuccessful min-cut run
- Max-Flow Min-Cut Theorem
- Deterministic Min-Cut Algorithm
- Stoer–Wagner minimum cut algorithm
- Pseudocode
- Randomized Min-Cut Algorithm
- Why randomized algorithms
- Karger's Randomized Algorithm
- Pseudocode
- Comparison: Deterministic vs Randomized Min-Cut
- Theorem for min-cut set
- Python simulation instructions

---

# 2. Min-Cut Problem

## 2.1 Why Use Min-Cut?

Min-cut algorithms are used in various applications to solve problems related to:

- Network connectivity
- Reliability
- Optimization

Specific applications include:

**Network Design**

- Helps improve communication efficiency
- Used to find the minimum capacity cut

**Communication Networks**

- Helps understand vulnerability to failures
- Helps build robust and fault-tolerant communication networks

**VLSI Design**

- Useful for partitioning circuits into smaller components
- Reduces interconnectivity complexity

Reference mentioned in lecture:
Section 1.5, *Application: A Randomized Min-Cut Algorithm in Probability and Computing (2nd Edition)*

---

# 3. Definition of Min-Cut

## 3.1 Cut-Set

A cut-set in a graph is:

> A set of edges whose removal breaks the graph into two or more connected components.

## 3.2 Minimum Cut Problem

Given a graph $G = (V, E)$ with $n$ vertices:

> The minimum cut (min-cut) problem is to find a minimum cardinality cut-set in $G$.

Min-cut algorithms such as Karger's algorithm:

- Are random
- Can be sensitive to the initial choice of edges
- Contracting critical edges early may lead to finding a smaller cut

---

# 4. Edge Contraction

The main operation in the algorithm is **edge contraction**:

Definition:

> Edge contraction is an operation that removes an edge from a graph while simultaneously merging the two vertices.

Procedure:
When contracting an edge $(u, v)$:

- Merge vertices $u$ and $v$ into one vertex
- Eliminate all edges connecting $u$ and $v$
- Retain all other edges
- The resulting graph may have parallel edges but no self-loops

The diagram on pages 15–16 illustrates the contraction process and resulting merged vertex structure.

---

# 5. Successful and Unsuccessful Min-Cut Runs

## 5.1 Successful Min-Cut Run

Definition:

> A successful min-cut run refers to success in the outcome of an algorithm designed to find the minimum cut in a graph.

The example diagram on page 19 shows a sequence of contractions resulting in the correct minimum cut.

---

## 5.2 Unsuccessful Min-Cut Run

Definition:

> An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

The diagram on page 22 shows a contraction sequence that eliminates edges belonging to the minimum cut, resulting in an incorrect answer.

---

# 6. Max-Flow Min-Cut Theorem

Statement:

> In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut.

Definitions used in the theorem:

**Capacity of a cut**

- Sum of capacities of edges in the cut oriented from a vertex in $X$ to a vertex in $Y$

**Minimum cut**

- Cut with smallest possible capacity

**Minimum cut capacity**

- Capacity of the minimum cut

**Maximum flow**

- Largest possible flow from source $S$ to sink $T$

The flow network diagram on page 26 illustrates this relationship.

---

# 7. Deterministic Min-Cut Algorithm

## 7.1 Stoer–Wagner Minimum Cut Algorithm

Let $s$ and $t$ be two vertices of a graph $G$.
Let $G/\{s, t\}$ be the graph obtained by merging $s$ and $t$.

Result:

> A minimum cut of $G$ can be obtained by taking the smaller of:

- A minimum $s$-$t$ cut of $G$, and
- A minimum cut of $G/\{s, t\}$.

Justification stated in lecture:

- Either a minimum cut separates $s$ and $t$, in which case the minimum $s$-$t$ cut is a minimum cut of $G$,
- Or none separates them, in which case a minimum cut of $G/\{s, t\}$ is the minimum cut.

## 7.2 Stoer–Wagner Algorithm Procedure (from slide)

Algorithm outline (as shown in the slide diagram):

1. While there is more than one vertex:
    - Find a cut in the current graph in the “natural way”
    - Make the graph smaller

Illustrative graph example is shown on page 28.

---

## 7.3 Pseudocode for Deterministic Min-Cut

### Algorithm 1: MinimumCutPhase(G, a)

```
pgsql

A ← {a};
while A ≠ V do
   add to A the most tightly connected vertex;
return the cut weight as the “cut of the phase”;
```

### Algorithm 2: MinimumCut(G)

```
yaml

while |V| ≥ 1 do
   choose any a from V;
   MinimumCutPhase(G, a);
   if the cut-of-the-phase is lighter than the current minimum cut then
      store the cut-of-the-phase as the current minimum cut;
   shrink G by merging the two vertices added last;
return the minimum cut;
``` :contentReference[oaicite:27]{index=27}

---

# 8. Randomized Min-Cut Algorithm

## 8.1 Why Randomized Algorithms?

Randomized algorithms:
```
```

- Provide a probabilistic guarantee of success
- Provide a more accurate estimate of the minimum cut with fewer iterations
:contentReference[oaicite:28]{index=28}

Additional properties listed:
- Efficiency
- Parallelization
- Approximation guarantees
- Avoidance of worst-case instances
- Heuristic nature
- Robustness :contentReference[oaicite:29]{index=29}

---

## 8.2 Karger's Randomized Algorithm

The example diagram on page 34 shows:
- Random selection of edges
- Edge contraction steps
- An example where the output cut is not minimal due to random choices
:contentReference[oaicite:30]{index=30}

---

## 8.3 Pseudocode: Recursive Randomized Min-Cut

**Algorithm 3: Recursive-Randomized-Min-Cut(G, α)**

Input:
- Undirected multigraph \(G\) with \(n\) vertices
- Integer constant \(α > 0\)

Output:
- A cut \(C\) of \(G\)

Procedure:

if n ⩽ α^3 then
C ← a min-cut of G found using brute force (exhaustive) search;
else

for i ← 1 to α do

G' ← multigraph obtained by applying n − ⌈n/√α⌉ random contraction steps on G;

C' ← Recursive-Randomized-Min-Cut(G', α);

if i = 1 or |C'| < |C| then

C ← C';

return C;

:contentReference[oaicite:31]{index=31}

---

# 9. Comparison: Deterministic vs Randomized Min-Cut

Choice of approach:
- Depends on the specific problem :contentReference[oaicite:32]{index=32}

### Deterministic Min-Cut
- Guarantees exact minimum cut
- May have higher time complexity for large graphs
- Stoer–Wagner time complexity:

$$
O(V \cdot E + V^2 \log V)
$$

:contentReference[oaicite:33]{index=33}

### Randomized Min-Cut
- Provides approximate minimum cut with high probability
- Karger’s algorithm time complexity:

$$
O(V^2)
$$

:contentReference[oaicite:34]{index=34}

---

# 10. Theorem for Min-Cut Set

Statement:
> The algorithm outputs a min-cut set with probability at least

$$
\frac{2}{n(n-1)}
$$

\]
:contentReference[oaicite:35]{index=35}

---

# 11. Python Simulation (Class Activity)

Instructions given in lecture:
- Open Campuswire post regarding Lecture 10
- Download the `.ipynb` file from comments :contentReference[oaicite:36]{index=36}

---

# 12. Summary of Key Concepts for Revision

For exam preparation, the following are central topics:
- Definition of cut-set and min-cut
- Edge contraction operation
- Successful vs unsuccessful runs
- Max-Flow Min-Cut theorem and definitions
- Stoer–Wagner algorithm concept and pseudocode
- Recursive randomized min-cut pseudocode
- Comparison of deterministic vs randomized approaches
- Probability theorem for Karger's algorithm

---

If you want, I can now **prepare a condensed 2-page "exam revision sheet" version of Lecture 10** (formulas, algorithms, and theorems only, optimized for last-minute study).